

游戏端日志数据接入规范

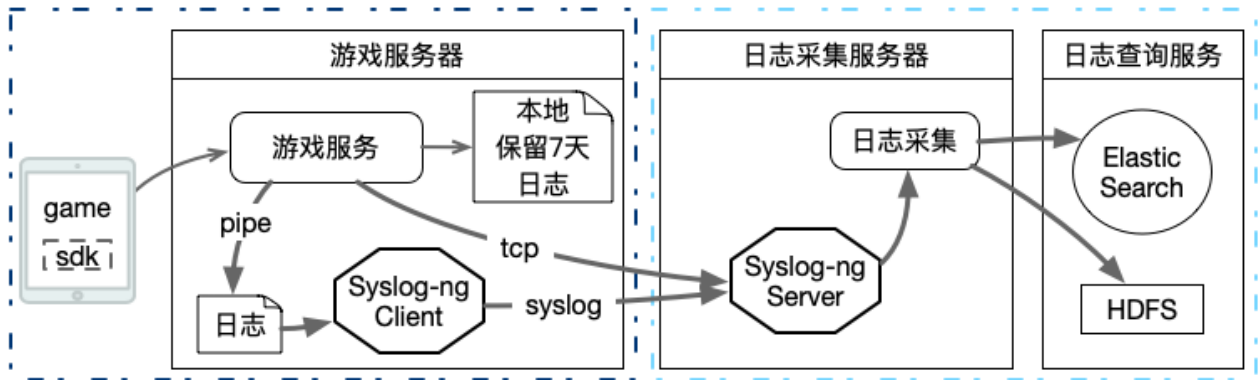
说明

本文档规范了游戏端日志数据采集接入到数据服务端的技术方案

原则

- 1. 所有日志均为实时发送；
- 2. 所有日志必须符合一定的格式规格；
- 3. 游戏端日志需要在本地保存一段时间（至少7天）；

日志数据采集逻辑



日志规格

单条日志格式如下：

<u>event time</u>	<u>area id</u>	<u>account id</u>	<u>role level</u>	platform	

newbie_guide_log 2019-11-20 00:00:00 p0001 1001 1091 568828186 9876212305116 21 A001 1 215					

struct name	<u>game id</u>	<u>group id</u>	<u>role id</u>	<u>channel id</u>	<u>step id</u>

日志的头部使用Struct Name标识，此标识描述某一类日志的集合；

event_time为日志时间；

之后的字段为具体的日志内容。

所有以上的字段用竖线 | 进行分割，并且日志采用UTF-8编码。

在日志中不得包含竖线 | 以及换行符 \n

日志发送

对于日志发送，有以下要求

1. 日志实时发送；
2. TCP发送单条日志长度上限为65536字节，字段最大长度为32K；
3. 由于数据采集过程中会出现网络异常等不可抗力因素，为了保证日志数据的完整性；
 1. 要求实现日志在本地备份缓存一段时间（至少7天）；
 2. 备份缓存文件必须是按照每天一个文件方式存放；
 3. 另外还需要实现在网络恢复正常后的数据重发功能来保证数据的完整性；
 4. 在发生错误时，需要在本地记录下错误原因以及重传内容的日志文件；
4. 每一分钟必须发送在线人数日志数据（可以看作是心跳数据）。

日志采集端采用的是Syslog-ng来收集日志。而日志的发送方式有两种：

1. 游戏服务器使用Linux系统，通过Syslog-ng客户端向Syslog-ng服务端发送日志；
2. 游戏服务器使用Window系统，采用TCP直连Syslog-ng服务端方式发送日志；

Syslog-ng 客户端发送日志

在游戏服务器端，部署Syslog-ng客户端来收集日志，并发送到日志采集服务器上部署的Syslog-ng服务端。

Syslog-ng提供了高性能的日志服务，实现了失败重发，故障重连，临时缓存等失效处理机制。

Syslog-ng客户端部署

安装

```
1  #在CentOS7中安装Syslog
2
3  #先卸载默认安装的rsyslog
4  $yum erase rsyslog
5
6  # 替换使用阿里的YUM源
7  $cd /etc/yum.repos.d/
8  $mkdir back
9  $mv *.repo back
10 $curl -o /etc/yum.repos.d/CentOS-Base.repo
    http://mirrors.aliyun.com/repo/Centos-7.repo
11 $curl -o /etc/yum.repos.d/epel.repo http://mirrors.aliyun.com/repo/epel-
    7.repo
12
13 # 安装syslog-ng
14 $yum install -y syslog-ng
```

启停服务

```
1 # 启动|重启|停止syslog-ng
2 $systemctl start syslog-ng
3 $systemctl restart syslog-ng
4 $systemctl stop syslog-ng
5 # 将syslog-ng加入自启动
6 $systemctl enable syslog-ng
```

配置

Syslog包含以下几个要素

1. Sources - 日志源位置
2. Filters - 日志过滤器
3. Destinations - 日志放置的位置（本地文件或远程主机）
4. Logs - 集合sources、filters和destinations的日志规则

配置文件位置：/etc/syslog-ng/syslog-ng.conf

重新加载配置：systemctl reload syslog-ng.service

```
1 @version:3.5
2 @include "scl.conf"
3
4 @module syslogformat
5 @module afsocket
6 @module affile
7 @module afprog
8 @module afuser
9 @module basicfuncs
10 @module dbparser
11 @define autoloading-modules 1
12 # 全局配置
13 options {
14     log_msg_size(671088640);
15     flush_lines(1);
16     log_fifo_size(1000000);
17     time_reopen(10);
18     use_dns(no);
19     dns_cache(yes);
20     use_fqdn(yes);
21     keep_hostname(yes);
22     chain_hostnames(no);
23     check_hostname(yes);
24     create_dirs(yes);
25     dir_perm(0755);
26     perm(0644);
27     stats_freq(1800);
28     mark_freq(0);
29     threaded(yes);
30 };
```

```

31
32 # 模版配置
33 template log_format {
34     template("$PROGRAM $MSG\n");
35     template_escape(no);
36 };
37
38 # 本地源配置
39 source s_log {
40     pipe("/home/log/syslog_pipe.log");
41 };
42
43 # 过滤器配置
44 filter f_level {
45     "${LEVEL_NUM}" > "5"
46 };
47
48 # 目的存放配置
49 destination d_log_remote {
50     tcp('10.1.10.196' port(5240) template(log_format));
51 };
52
53 # 调用上面配置
54 log {
55     source(s_log);
56     filter(f_level);
57     destination(d_log_remote);
58     flags(final);
59 };

```

1. 以上模版仅为样例，请根据实际情况修改配置
2. 不同版本的配置项可能不同，以下列出部分配置项说明；可参见附录或者参考Syslog-ng官方文档

日志发送例程

假设日志格式如下：

```

1 | newbie_guide_log|2019-11-20
   | 00:00:04|p0001|1001|1091|568828186|9876212305116|21|A001|1|215

```

在Java中通过Syslog-ng客户端发送日志的逻辑如下：

1. 通过Java的FILE IO将日志数据写入配置的pipe文件中（例子中为：/home/log/syslog_pipe.log）
2. 需要在日志结尾添加 \n

写日志文件采用Guava的CharSink（Append模式），这种方式在多线程中也是适用的。

```

1 <dependency>
2   <groupId>com.google.guava</groupId>
3   <artifactId>guava</artifactId>
4   <version>28.0-jre</version>
5 </dependency>

```

```

1 import com.google.common.io.CharSink;
2 import com.google.common.io.FileWriteMode;
3 import com.google.common.io.Files;
4
5 import java.io.File;
6 import java.io.IOException;
7
8 public class LogSender {
9
10     public static void main(String[] args) {
11         try {
12             String content = "newbee_guide_log|2019-11-20
13 00:00:04|p0001|1001|1091|568828186|9876212305116|21|A001|1|215" + "\n";
14             CharSink bw = Files.asCharSink(new
15 File("/home/log/syslog_pipe.log"), Charsets.UTF_8, FileWriteMode.APPEND);
16             bw.write(content);
17         } catch (IOException e) {
18             e.printStackTrace();
19         }
20     }
21 }

```

Java发送日志

如果不采用Syslog-ng，可以采用Java直接TCP方式发送日志。

假设生成的日志为：

```

1 newbee_guide_log|2019-11-20
  00:00:04|p0001|1001|1091|568828186|9876212305116|21|A001|1|215

```

Java 方式通过TCP实现发送日志的逻辑如下

```

1 import java.io.IOException;
2 import java.io.PrintWriter;
3 import java.net.Socket;
4
5 public class SendTcp {
6     public static void main(String[] args) throws IOException {

```

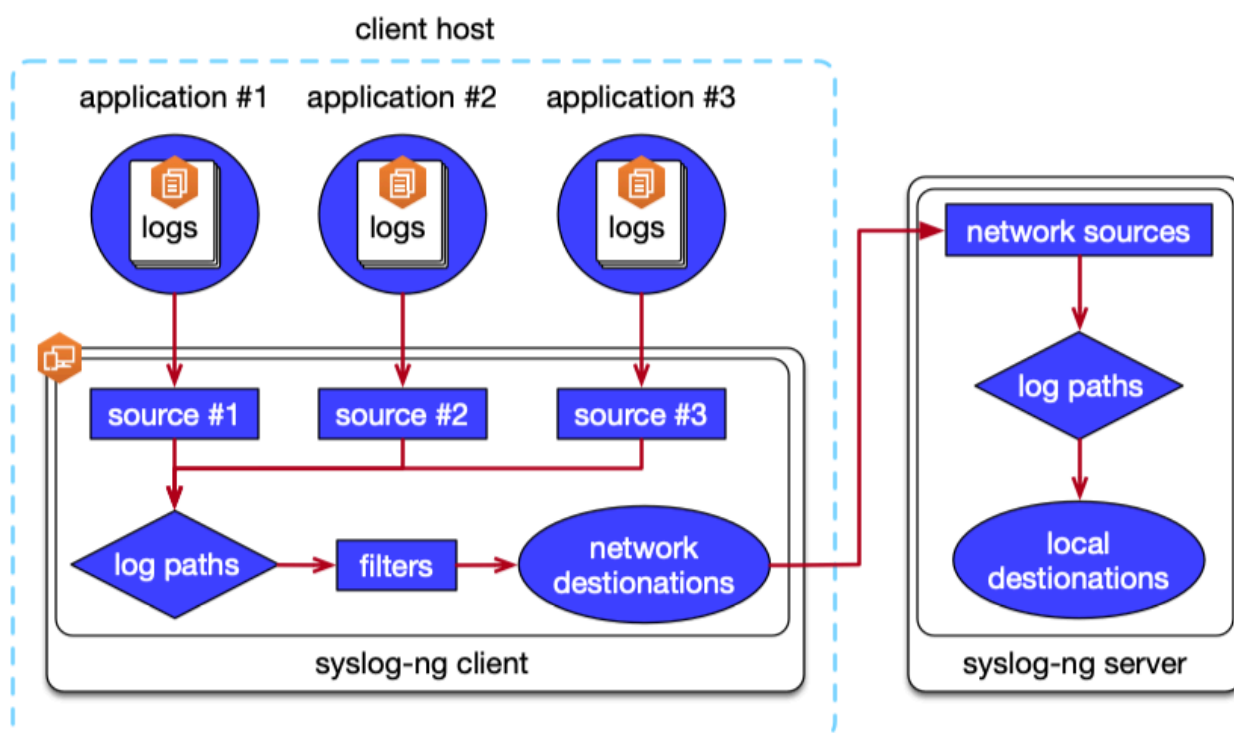
```

7      Socket s = null;
8      PrintWriter pw = null;
9      try {
10         s = new Socket("116.211.20.206", 5240);
11         pw = new PrintWriter(s.getOutputStream(), true);
12         String content = "newbee_guide_log|2019-11-20
00:00:04|p0001|1001|1091|568828186|9876212305116|21|A001|1|215" + "\n";
13         pw.write(content);
14     } catch (IOException e) {
15         e.printStackTrace();
16     } finally {
17         pw.flush();
18         pw.close();
19         s.close();
20     }
21 }
22 }
23

```

附录

Syslog-ng工作原理



Syslog-ng配置项

全局配置项说明

- 1 `chain_hostnames(yes|no)` # 是否打开主机名链功能，打开后可在多网络段转发日志时有效

```

2 long_hostnames(yes|no) # 是chain_hostnames的别名, 已不建议使用
3 keep_hostname(yes|no) # 是否保留日志消息中保存的主机名称
4 use_dns(yes|no) # 是否打开DNS查询功能,
5 use_fqdn(yes|no) # 是否使用完整的域名
6 check_hostname(yes|no) # 是否检查主机名有没有包含不合法的字符
7 bad_hostname(regex) # 可通过正规表达式指定某主机的信息不被接受
8 dns_cache(yes|no) # 是否打开DNS缓存功能
9 dns_cache_expire(n) # DNS缓存功能打开时, 一个成功缓存的过期时间
10 dns_cache_expire_failed(n) # DNS缓存功能打开时, 一个失败缓存的过期时间
11 dns_cache_size(n) # DNS缓存保留的主机名数量
12 create_dirs(yes|no) # 当指定的目标目录不存在时, 是否创建该目录
13 dir_owner(uid) # 目录的UID
14 dir_group(gid) # 目录的GID
15 dir_perm(perm) # 目录的权限, 使用八进制方式标注, 例如0644
16 owner(uid) # 文件的UID
17 group(gid) # 文件的GID
18 perm(perm) # 文件的权限, 同样, 使用八进制方式标注
19 gc_busy_threshold(n) # 当syslog-ng忙时, 其进入垃圾信息收集状态的时间一旦分派的对象
    达到这个数字, syslog-ng就启动垃圾信息收集状态。默认值是: 3000。
20 gc_idle_threshold(n) # 当syslog-ng空闲时, 其进入垃圾信息收集状态的时间一旦被分派的
    对象到达这个数字, syslog-ng就会启动垃圾信息收集状态, 默认值是: 100
21 log_fifo_size(n) # 输出队列的行数
22 log_msg_size(n) # 消息日志的最大值 (bytes)
23 mark(n) # 多少时间(秒)写入两行MARK信息供参考, 目前没有实现
24 stats(n) # 多少时间(秒)写入两行STATUS信息, 默认值是: 600
25 sync(n) # 缓存多少行的信息再写入文件中, 0为不缓存, 局部参数可以覆盖该值。
26 time_reap(n) # 在没有消息前, 到达多少秒, 即关闭该文件的连接
27 time_reopen(n) # 对于死连接, 到达多少秒, 会重新连接
28 use_time_recvd(yes|no) # 宏产生的时间是使用接受到的时间, 还是日志中记录的时间; 建议
    使用R_的宏代替接收时间, S_的宏代替日志记录的时间, 而不要依靠该值定义。

```

消息源配置项说明

```

1 格式: source <sourcename> {sourcedriver params; sourcedriver params; ... };
2 internal() # syslog-ng内部产生的消息
3 file() # 从指定的文件读取日志信息
4 pipe() # 从指定的管道, 读取日志信息
5 fifo() # 从指定的FIFO设备, 读取日志信息
6 program() # 打开指定的应用程序, 从它的标准输出读取消息
7 sun-stream(), sun-streams() # 在solaris系统中, 打开一个(多个)指定的STREAM设
    备, 从其中读取日志消息
8 tcp(), tcp6() # 在指定的TCP端口接收日志消息
9 udp(), udp6() # 在指定的UDP端口接收日志消息
10 unix-dgram() # 打开指定的SOCK_DGRAM模式的unix套接字, 接收日志消息
11 unix-stream() # 打开指定的SOCK_STREAM模式的unix套接字, 接收日志消息

```

过滤器配置项说明

```
1 格式: filter <filtername> {expression; };
2  facility() # 根据facility (设备) 选择日志消息
3  filter() # 调用另一条过滤规则
4  host() # 日志消息的主机名是否和一个正则表达式匹配
5  level() or priority() # 根据level (优先级) 选择日志消息
6  match() # 对日志消息的内容进行正则匹配
```

消息目的地配置项说明

```
1 格式: destination <destname> {destdriver params; destdriver params; ... ;
   };
2  file() # 把日志消息写入指定的文件
3  pipe() # 把日志消息发送到指定的管道
4  fifo() # 把日志消息发送到指定的FIFO设备
5  program() # 启动指定的程序, 并把日志消息发送到该进程的标准输入
6  sql() # 把日志消息写入数据库, 适用于3.x版本及更高版本的syslog-ng
7  tcp() and tcp6() # 把日志消息发送到指定的TCP端口
8  udp() and udp6() # 把日志消息发送到指定的UDP端口
9  unix-dgram() # 把日志消息写入指定的SOCK_DGRAM模式的unix套接字
10 unix-stream() # 把日志消息写入指定的SOCK_STREAM模式的unix套接字
11 usertty() # 把日志消息发送到已经登陆的指定用户终端窗口
```

消息路径配置

```
1 格式: log { source S1; source S2; ...filter F1; filter F2; ... destination
   D1; destination D2; ... };
```